

Guion de Exposición — EE1: Yunnan Lucky Air

Este documento contiene la estructura y redacción final para la sustentación del proyecto, excluyendo referencias a errores y enfocado en los logros técnicos y el flujo de trabajo moderno.

PARTE 1 — Navegación y Propuesta Funcional (Iván)

Objetivo: Mostrar el producto en vivo y demostrar la solución de negocio.

1. Contexto de Negocio:

- o Lucky Air es una aerolínea regional de Yunnan, China, que necesitaba digitalizarse para captar tanto pasajeros comunes como empresas. Al no tener los requerimientos exactos de un cliente físico, el equipo analizó la problemática e ideó completamente esta solución funcional.

2. Navegación Interactiva:

- o Recorrido por el menú "Hero" (buscador de vuelos).
- o Exposición de destinos turísticos de Yunnan (Dali, Xishuangbanna).
- o Paseo por las distintas áreas web de los segmentos: Estudiantes, Corporativo, Reseñas y Autoservicio.

```
graph TD
    %% Patrón de Colores UX
    classDef hero fill:#DBEAFE,stroke:#3B82F6,stroke-width:2px,color:#1E3A8A;
    classDef page fill:#F8FAFC,stroke:#94A3B8,stroke-width:2px,color:#0F172A;

    Home[Inicio - index.html]:::hero
    Home --> Est(Estudiantes - students):::page
    Home --> Corp(Corporativo - corporate):::page
    Home --> Com(Comunidad - community):::page
    Home --> Auto(Autoservicio - self-service):::page
    Home --> Reg(Auth - login / register):::page
```

3. Página en Vivo: - URL de GitHub Pages: <https://ucal-systems-engineering-1st.github.io/yunnan-lucky-air/>

4. Despliegue Automático (Deploy CI/CD):

- o Explicar brevemente que todo el ecosistema está vinculado a GitHub Pages. De este modo el entorno se despliega y actualiza de forma automática con cada nuevo cambio que el equipo aprueba hacia la rama principal.
-

PARTE 2 — Repositorio, GitFlow (Geraldo)

Objetivo: Demostrar el orden organizativo del equipo, la automatización y el nivel de ingeniería empleado.

1. Apertura del Repo y Arquitectura GitFlow:

- o URL: `https://github.com/ucal-systems-engineering-1st/yunnan-lucky-air`
- o "Para evitar sobrescribir el trabajo de los compañeros y mantener un orden estrictamente profesional, adoptamos el patrón arquitectónico GitFlow. Este modelo divide nuestro flujo de trabajo productivo en 5 ramas especializadas:"
 - **main** : El código más estable en producción (Lo que ve el usuario final de forma remota).
 - **hotfix** : Ramas de emergencia técnica. Nacen desde **main** para hacer reparaciones críticas rápidas (ej. arreglo de imágenes locales).
 - **release** : Ramas "puente". Preparan y pulen el código de desarrollo en general antes de lanzarlo con firmeza y seguridad hacia la rama master.
 - **develop** : Nuestro entorno principal de integración. Aquí confluye a diario el nuevo código que creamos independientemente.
 - **feature** : Las ramificaciones de trabajo individual aisladas (para que cada quien programe lo suyo sin romper el código ajeno).

```
%%{init: { 'logLevel': 'debug', 'theme': 'base', 'gitGraph': {'showBranches': true}

    gitGraph
        commit

        branch hotfix
        branch release
        branch develop
        branch feature

        checkout feature
        commit
        commit

        checkout develop
        merge feature

        checkout release
        merge develop
        commit

        checkout main
        merge release tag: "v1.0.0"

        checkout hotfix
        commit

        checkout main
        merge hotfix tag: "v1.0.1"
    }
```

2. La Cultura del Pull Request (Code Review):

- o "Establecimos una política muy clara de Pull Requests dentro del equipo. Ningún integrante podía inyectar o empujar su código directamente. Todo código nuevo pasó por el proceso de un PR para lograr ser verificado o aprobado, protegiendo así nuestro despliegue."

3. Integración Continua (CI) e inspección de Accesibilidad:

- Ir a la pestaña "Actions" y mostrar los flujos de checks de color verde.
- *"Finalmente logramos estandarizar calidad usando Integración Continua (CI) en GitHub Actions. Lo configuramos estructuralmente para que actúe como nuestro 'Linter' estricto: exigía el uso fundamental de accesibilidad y parámetros web (como atributos obligatorios de tipo `alt` al renderizar imágenes, o campos correctos de tipo `<Label>`). Si la vista de un compañero no superaba estos duros estándares de accesibilidad, el CI abortaba la prueba y bloqueaba automáticamente el pase a producción, lo que nos forzó a programar a todos siempre con la mayor higiene y disciplina posible."*

```
graph LR
    A[Pull Request Grupal] --> B[GitHub Actions Linter / CI]
    B --> C[Valida Sintaxis HTML5]
    C --> D[Comprueba Accesibilidad]
    D --> E[Merge Desbloqueado]
```

```
flowchart LR
    %% Estilos UX/UI Limpios
    classDef actor fill:#F8FAFC,stroke:#CBD5E1,stroke-width:2px,color:#334155;
    classDef system fill:#DBEAFE,stroke:#3B82F6,stroke-width:2px,color:#1E3A8A;
    classDef success fill:#D1FAE5,stroke:#10B981,stroke-width:2px,color:#065F46;

    A[Pull Request Grupal]:::actor --> B[GitHub Actions Linter / CI]:::system
    B -->|Valida Sintaxis HTML5| C[Pipeline Aprobado ✓]:::success
    C -->|Comprueba Accesibilidad| D[ ]
    D --> E[Merge Desbloqueado]:::success
```

PARTE 3 — Dominio Técnico y Etiquetas HTML (Por Integrante)

Objetivo: Demostrar el uso correcto de HTML5 semántico y buenas prácticas de maquetación frente a los jurados.

1. Rachel Duarte (`index.html` , `login.html` , `register.html` , `views/self-service.html`)

- **Enfoque:** Creación de la portada principal (Landing Page), el sistema interactivo de autenticación y el portal integral de autogestión de usuarios.
- **Etiquetas Clave:**
 - **En `index.html`** : Uso maestro de `<header>` y `<nav>` combinadas con su estructuración de listas `<ul>` y `<li>` para consolidar el menú centralizado.
 - **En `index.html`** : Implementación semántica del Hero interactivo dentro de la etiqueta `<form>` , integrando datos obligatorios mediante `<input type="text">` , `<input type="date">` bajo marcaje `required` .
 - **En `login.html` y `register.html`** : Construcción del panel de accesos empleando campos `<input type="password">` y perfilamiento del turista agrupado por opciones usando `<select>` unido a `<option>` .
 - **En `views/self-service.html`** : Fragmentación limpia de las acciones del usuario (check-in, reclamos y viajes) usando múltiples contenedores independientes `<form>` .

2. Iván (`views/corporate.html`)

- **Enfoque:** Creación del despliegue masivo y servicios B2B destinados a clientes empresariales formales.
- **Etiquetas Clave (Todo trabajado en `views/corporate.html`):**
 - Uso intensivo del componente encapsulador `<section>` (ej. `<section class="benefits">`) para segmentar rígidamente por sectores de lectura del plano web.

- Empleo de diagramas y organización espacial manipulando libremente elementos divisores `<div>` como principales herramientas de la grilla principal.
- Creación y envío de datos uniendo semánticamente una variable literal `<label>` y asociándola firmemente hacia la entrada obligatoria `<input>` empleando atributos limpios de identificación (ID).

3. Alessandro (`views/community.html` y `views/students.html`)

- **Enfoque:** Sistema base de las Reseñas Comunitarias de viajeros y el Módulo de Paquetes o Beneficios Estudiantiles Universitarios.
- **Etiquetas Clave:**
 - **En `views/students.html` :** Despliegue arquitectónico empleando el elemento `<table>` y dividiendo de cara limpia tanto sus cabeceras primarias `<thead>` como el bucle iterativo de filas dentro de un robusto `<tbody>` .
 - **En `views/students.html` y `views/community.html` :** Iteración programada replicando modularmente testimonios sociales y beneficios educativos, acorazándolos en listas (`<ul>` y `<li>`) o bloques `<section>` .
 - **En `views/community.html` :** Dedicación integral hacia la accesibilidad web inyectando obligatoriamente descriptores alternativos `alt="Descripción"` en las etiquetas gráficas `<img>` , dando soporte a sistemas de narración de pantalla.